

GuardianIQ

Phase 2 UI Screen Design Developer Specification

Governed Workflow Scheduling & AI Agent Execution

Core UI design statement: Phase 2 screens must help users create, approve, schedule, execute, monitor and audit AI-agent workflows through governed experiences. The UI must not behave like a simple cron scheduler; every screen must make ownership, boundary rules, authorization, approvals, run traceability and audit status visible to the user.

Field	Value
Platform	GuardianIQ
Phase	Phase 2
Document Type	Detailed Frontend UI / UX Developer Specification
Primary Source	GuardianIQ Phase 2 Developer Specification - Governed Workflow Scheduling & AI Agent Execution
Pre-requisites	Phase 0 foundation and Phase 1 Governance Registry completed
Audience	Frontend developers, backend developers, QA, product owner, architect and PM

1. Executive Summary

This document converts the GuardianIQ Phase 2 developer specification into a detailed screen-by-screen UI implementation guide. It is intended to become the frontend base document for developers implementing governed workflow scheduling, AI agent assignment, run monitoring, approval handling, authorization simulation and audit visibility.

Important interpretation: The attached Phase 2 specification defines the target as a governed workflow execution capability, not a generic scheduler. The screens therefore must expose governance context: who owns the schedule, what AI agent/model is assigned, what tools/data are allowed, what execution mode is permitted, whether approval is required, what was executed, what outputs were generated, and what audit events were logged.

2. Source Scope Interpreted for UI

Source Spec Area	UI Interpretation
Workflow scheduling	The user needs a scheduler dashboard, creation wizard, schedule detail and lifecycle actions.
AI agent assignment	The UI must provide controlled assignment of registered AI agents, AI models, tools and data boundaries.
Execution boundaries	The UI must allow users to configure allowed tools, allowed data sources, blocked operations, runtime limits, retries and confidence thresholds.
Run tracking	The UI must show workflow runs, run status, run steps, outputs, evidence, failures and retry history.
Human oversight	The UI must clearly show approval requirement, approval queue, approver decision and escalation path.
Audit-first design	Schedule detail and run detail must include audit timeline panels and event traceability.
RBAC + ABAC	Buttons must be hidden/disabled based on permission and backend authorization decision; sensitive actions require authorization check.

3. UI Design Principles

ID	Principle	Developer Implication
UI-P1	Governance-first, not scheduler-first	Every scheduling screen must surface owner, risk, approval, boundary and audit context.
UI-P2	Backend authorization is authoritative	Frontend may hide buttons for UX, but all sensitive actions must still call backend authorization.
UI-P3	Wizard-based guided creation	Schedule creation must be step-by-step to prevent invalid or unsafe configuration.
UI-P4	Explain why approval is required	High-risk, write-capable, sensitive-tool or critical schedules must show approval reason banners.
UI-P5	Trace every execution	Run detail must show chronological run steps, status, outputs and audit events.
UI-P6	Never expose sensitive output by default	Run outputs must respect sensitivity and authorization scope; mask or hide as needed.
UI-P7	Design for failure	Failures, retries, skipped runs, boundary violations and authorization denials must be first-class UI states.
UI-P8	Use registry-driven dropdowns	Workflow, AI agent, AI model, tools, data sources, owner, approval group and departments must come from backend APIs.

4. Information Architecture and Route Map

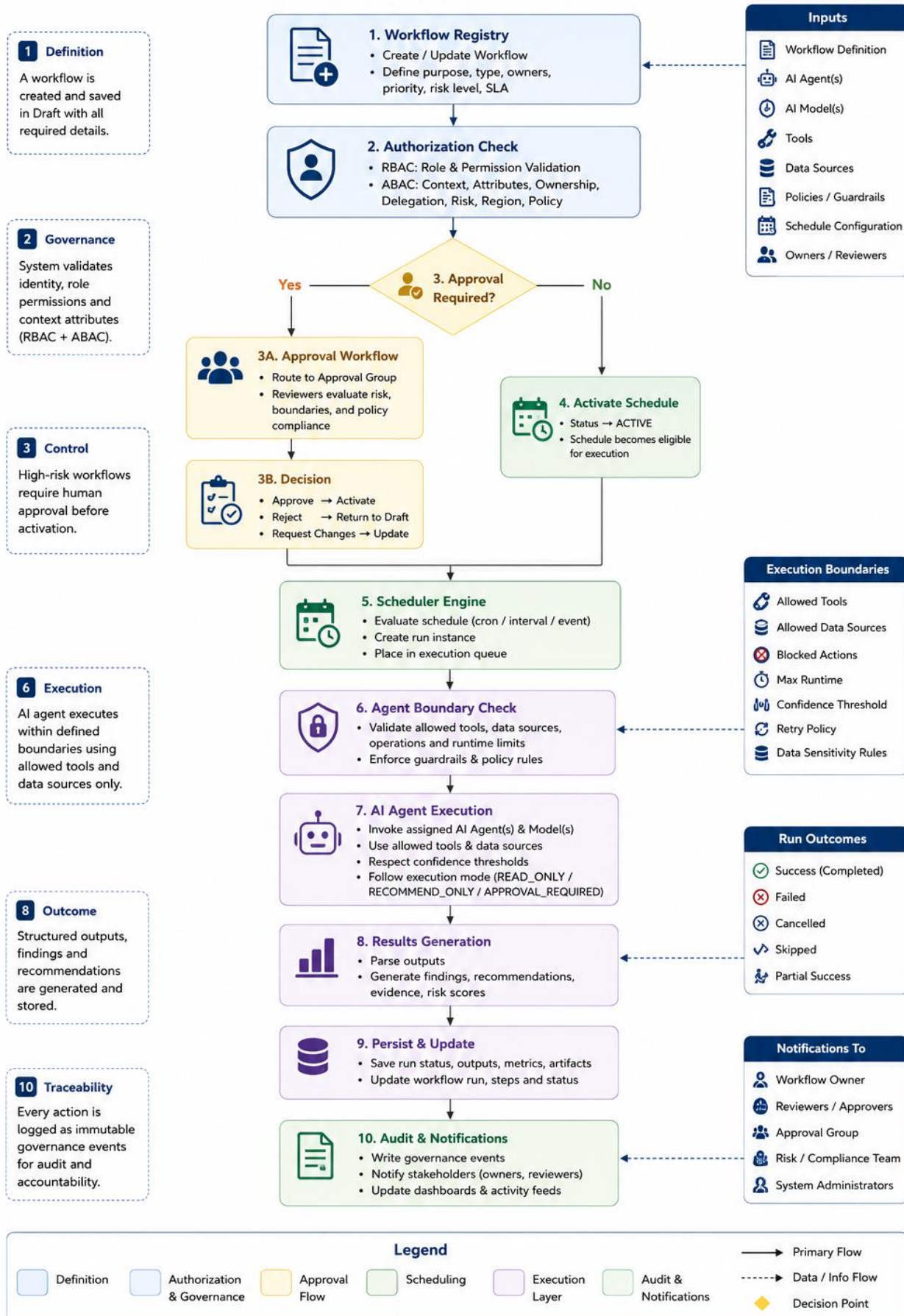
Route	Screen	Primary Purpose	Data Dependency
/workflow-scheduler	Workflow Scheduler Dashboard	List, filter and manage schedules.	workflow_schedules + registry.workflows
/workflow-scheduler/new	Create Schedule Wizard	Create governed schedule through controlled steps.	registry workflows, agents, models, tools, data sources, users, approval groups
/workflow-scheduler/:id	Schedule Detail	View configuration, lifecycle, approvals, assignments, runs and audit events.	workflow_schedules, assignments, approvals, runs, audit events
/workflow-runs	Run History	Search/filter all workflow runs.	workflow_runs
/workflow-runs/:runId	Run Detail	Full run trace, steps, outputs, failures and audit timeline.	workflow_runs, run_steps, outputs, failures, audit events
/agent-assignments	Agent Assignment Matrix	Manage or review workflow-to-agent/model/tool relationships.	workflow_schedule_agent_assignments + registry tables

/authorization-simulator	Authorization Simulator	Test RBAC + ABAC decision before sensitive operation.	authorization_decisions + core identity + registry context
/schedule-approvals	Schedule Approval Queue	Approve/reject schedule activation requests.	workflow_schedule_approvals
/workflow-notifications	Notifications Center	View approval, failure, SLA and high-risk output notifications.	workflow_notifications

5. Navigation and Menu Placement

1. End-to-End Workflow Execution Flow

From Schedule Creation to Execution, Output and Audit



Menu Group	Menu Item	Route	Visible To
Governance Execution	Workflow Scheduler	/workflow-scheduler	Governance Admin, AI Asset Owner, Risk Manager, Auditor with view rights
Governance Execution	Run History	/workflow-runs	Governance Admin, AI Reviewer, Auditor, AI Asset Owner
Governance Execution	Schedule Approvals	/schedule-approvals	Approval group members, Risk Manager, Compliance Officer
Governance Configuration	Agent Assignments	/agent-assignments	Governance Admin, AI Asset Owner
Governance Tools	Authorization Simulator	/authorization-simulator	System Admin, Governance Admin, Security/Admin roles
Alerts	Workflow Notifications	/workflow-notifications	All authorized users with assigned notifications

6. Shared UI Components and Standards

6.1 Component Inventory

Component	Purpose	Used On	Key Props / Data
ScheduleStatusBadge	Display DRAFT, PENDING_APPROVAL, ACTIVE, PAUSED, RETIRED, FAILED.	Dashboard, Detail, Approval Queue	status, riskLevel, isOverdue
RiskLevelBadge	Display LOW/MEDIUM/HIGH/CRITICAL consistently.	All schedule/run screens	riskLevel, showIcon
ApprovalRequirementBanner	Explain why approval is required.	Create Wizard, Schedule Detail	approvalRequired, approvalReasons, approverGroup
AgentAssignmentPanel	Display assigned agent, model, execution mode and boundaries.	Wizard, Detail, Assignment Matrix	agent, model, executionMode, tools, dataSources
BoundaryRuleEditor	Edit allowed tools, data sources and blocked operations.	Wizard Step 3, Assignment Matrix	allowedTools, allowedDataSources, blockedOperations
CronExpressionBuilder	Build schedule timing without forcing users to write cron manually.	Wizard Step 4	scheduleType, cronExpression, timezone
RunTimeline	Show run steps in sequence with timestamps and status.	Run Detail	runSteps, auditEvents, failures
RunOutputViewer	Show findings, recommendations, evidence and raw output if authorized.	Run Detail	outputs, sensitivity, canViewRaw
AuthorizationDecisionPanel	Show Allow/Deny decision and reasons.	Authorization Simulator, Action confirmations	decision, reasons, evaluatedAttributes
AuditTimelinePanel	Show immutable audit events for schedule/run.	Schedule Detail, Run Detail	entityType, entityId, events
EmptyStateCard	Standard empty state with action.	All list screens	title, message, actionText, actionHandler
ErrorStatePanel	Display recoverable/non-recoverable errors.	All screens	errorCode, message, retryAction

6.2 Global Field Rules

Field Type	Rule
UUID References	Never typed manually. Always selected from backend-driven dropdown or hidden route param.
Status	Use controlled badges and enum mapping; never display raw technical enum without label.
Risk Level	Show text and visual indicator; high/critical must trigger explanation.
Timezone	Default to tenant timezone but must be visible and editable during schedule creation.
Cron Expression	Advanced input may be shown, but friendly builder must be the default.
Metadata JSON	Hide from business users. Show only under Advanced section for admin/dev roles.
Audit Events	Read-only, chronological, with actor, event, timestamp and summary.

6.3 Standard Page States

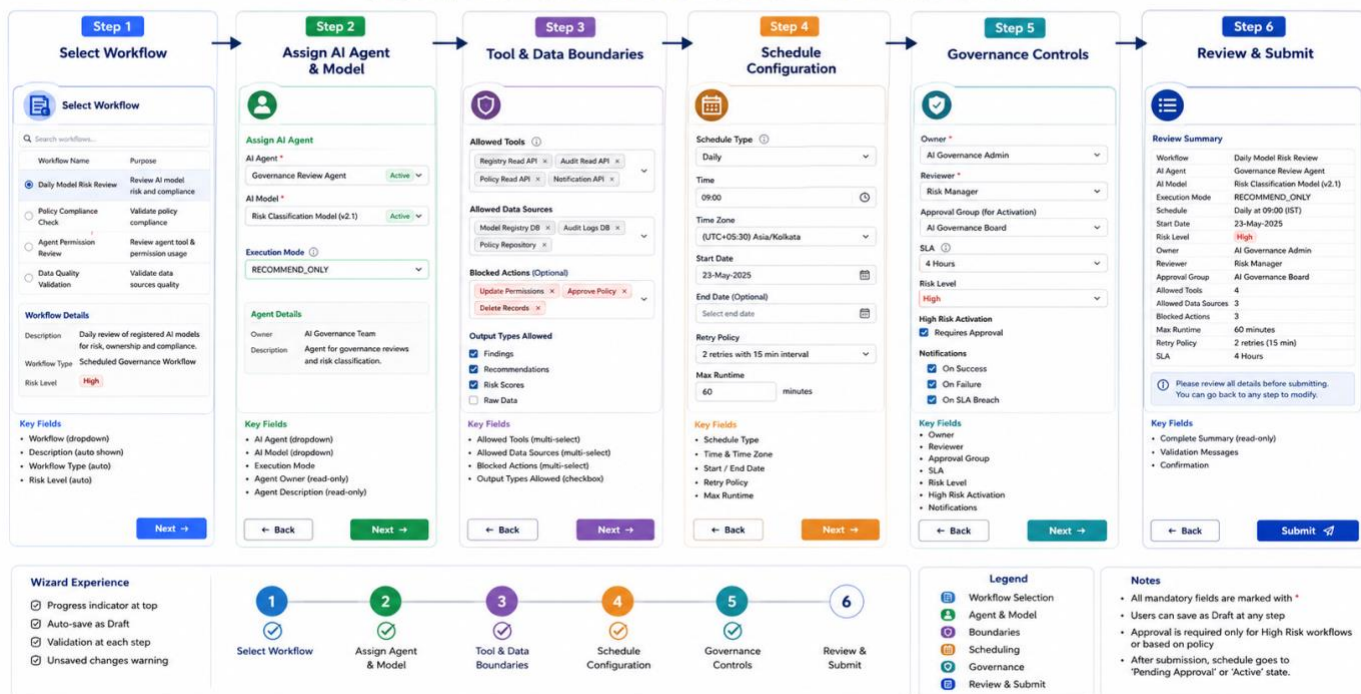
State	Required UX Behavior
Loading	Show skeleton table/cards; avoid blank page.
Empty	Explain what is missing and provide next action if user has permission.
Error	Show user-friendly error, request ID, retry option and escalation hint where required.
Unauthorized	Show restricted access message; do not reveal sensitive object details.
Partial Access	Show record but mask restricted output sections.
Validation Failure	Show field-level errors and top summary.
Action Confirmation	Use confirmation modal for activate, pause, resume, retire, cancel, run-now and reject.

7. Workflow Scheduler Dashboard

Purpose: Provide an operational command view of all governed workflow schedules, their status, risk, next run time, owner, approval state and health.

2. Create Schedule Wizard Screen Flow

Step-by-Step UI Flow to Create and Activate a Governed Workflow Schedule



Item	Specification
Route	/workflow-scheduler
Primary users	Governance Admin, AI Asset Owner, Risk Manager, Compliance Officer, Auditor
Core outcome	User can understand which schedules exist, what is active, what needs approval, what failed and what can be acted upon.
Primary backend dependency	GET /api/v1/workflow-scheduler/schedules
Key governance rule	Records outside the user scope must be hidden or restricted by backend authorization.

Recommended Layout

- Top summary KPI cards: Total Schedules, Active, Pending Approval, Failed/Attention, High Risk.
- Filter bar: Status, Risk Level, Owner, Workflow, Agent, Department, Schedule Type, Next Run Range.
- Main paginated table with sortable columns and row-level actions.
- Right-side optional quick detail drawer when a row is selected.
- Primary action button: Create Schedule, visible only if CREATE_WORKFLOW_SCHEDULE is allowed.

Fields, Controls and Data Binding

UI Area / Field	Control Type	Source / API	Validation / Behavior
Search	Text input	query param	Search schedule name, code, workflow, owner.
Status Filter	Multi-select	lookup/status	Default: Active + Pending Approval + Failed.
Risk Filter	Multi-select	lookup/risk_level	High/Critical should appear first.

Schedule Table	Data table	GET schedules	Must support pagination, sorting and server filters.
Next Run At	Date/time display	workflow_schedules.next_run_at	Display timezone-aware value.
Health Column	Computed badge	runs + failures	Show Healthy, Attention, Failed, SLA Breach.

Actions and Button Rules

Action	Visible To	Backend Call	Enable / Disable Rule	Result
Create Schedule	CREATE_WORKFLOW_SCHEDULE	navigate /new	User has create permission.	Wizard opens.
Open Detail	VIEW_WORKFLOW_SCHEDULE	GET schedule detail	User can view schedule.	Detail page opens.
Run Now	RUN_WORKFLOW_SCHEDULE	POST /run-now	Schedule ACTIVE and backend allows.	Run created.
Pause	PAUSE_WORKFLOW_SCHEDULE	POST /pause	Schedule ACTIVE.	Status PAUSED.
Resume	RESUME_WORKFLOW_SCHEDULE	POST /resume	Schedule PAUSED.	Status ACTIVE.
Retire	RETIRE_WORKFLOW_SCHEDULE	POST /retire	Not already RETIRED.	Status RETIRED.

UI States

State	User Experience
No schedules	Show message: No governed schedules configured. If authorized, show Create Schedule.
Pending approvals	Rows should show warning badge and link to approval queue.
Failed schedules/runs	Rows should show attention state with last failure reason.
Unauthorized action	Disable action and tooltip: You do not have permission or context to perform this action.

Screen Acceptance Criteria

- Dashboard must load paginated results and not fetch all schedules into frontend memory.
- High-risk and failed schedules must be visually distinguishable.
- Run Now, Pause, Resume and Retire must call backend authorization before execution.
- Table filters must be reflected in URL query params for shareability and reload consistency.

8. Create Schedule Wizard

Purpose: Guide users through safe creation of a governed workflow schedule linked to registered workflow, AI agent, model, tools, data sources, approval group and execution boundaries.

Item	Specification
Route	/workflow-scheduler/new
Primary users	Governance Admin, AI Asset Owner with create rights
Core outcome	Creates DRAFT or PENDING_APPROVAL schedule based on risk, execution mode and tool sensitivity.
Primary backend dependency	POST /api/v1/workflow-scheduler/schedules
Key governance rule	Wizard must prevent unsafe configuration but backend remains source of truth.

Recommended Layout

- Use a horizontal or vertical six-step wizard: Workflow, Agent/Model, Boundaries, Schedule, Governance, Review & Submit.
- Show completion state per step: Complete, Missing, Warning, Error.
- Show persistent right summary panel with risk level, approval requirement and selected assets.
- Allow Save Draft after minimum required Step 1 fields are present.
- Final submit must show validation summary before backend call.

Fields, Controls and Data Binding

UI Area / Field	Control Type	Source / API	Validation / Behavior
Step 1: Workflow	Dropdown/search	GET registry workflows	Workflow must be ACTIVE.
Workflow Criticality	Read-only badge	registry.workflows.criticality_level	Used for approval logic.
Step 2: Agent	Dropdown/search	GET registry agents	Agent must be ACTIVE and not deleted.
Model	Dropdown/search	GET registry models	Model must be active; optional if agent has default model.
Execution Mode	Dropdown	agent registry + lookup	Cannot exceed agent registered execution boundary.
Confidence Threshold	Numeric	form	0-100; show recommended default from agent.
Step 3: Allowed Tools	Multi-select	GET registry tools	Write/execute tools trigger approval.
Allowed Data Sources	Multi-select	GET registry data sources	Restricted sensitivity must trigger approval banner.
Blocked Operations	Multi-select/tags	lookup/operations	At least one blocked operation recommended for recommend-only.
Max Runtime	Number	form	Must be > 0; default 1800 seconds.
Step 4: Schedule Type	Dropdown	lookup schedule type	MANUAL, ONE_TIME, INTERVAL, DAILY, WEEKLY, MONTHLY, CRON.
Cron Expression	Builder + advanced input	form	Required only for CRON; validate before submit.
Timezone	Dropdown	tenant config/timezone list	Required; default tenant timezone.
Start / End Date	Date-time picker	form	End must be after start.
Concurrency Policy	Dropdown	lookup	SKIP_IF_RUNNING recommended default.
Retry Policy	Structured editor	form JSON	Max retries, retry delay, retry on failure types.
Step 5: Owner	User dropdown	GET users	Owner required.
Reviewer	User/group dropdown	GET users/groups	Required for high-risk.
Approval Group	Group dropdown	GET approval groups	Required if approval_required=true.
SLA	Number/time unit	form	Required if risk HIGH/CRITICAL.
Step 6: Review	Read-only summary	all prior data	Show validation warnings and approval reasons.

Actions and Button Rules

Action	Visible To	Backend Call	Enable / Disable Rule	Result
Next Step	Wizard user	client validation	Current step valid or warnings accepted.	Move forward.
Previous Step	Wizard user	none	Not on Step 1.	Move back.
Save Draft	CREATE_WORKFLOW_SCHEDULE	POST schedules as DRAFT	Minimum workflow + name + owner present.	Draft created.
Submit	CREATE/SUBMIT permission	POST schedules	All required fields valid.	DRAFT/PENDING_APPROVAL/ACTIVE returned.

Cancel	Wizard user	navigate back	Confirm if unsaved changes.	Exit wizard.
--------	-------------	---------------	-----------------------------	--------------

UI States

State	User Experience
Approval required	Show banner explaining whether caused by high risk, write tool, restricted data, critical workflow or execution mode.
Invalid registry item	If selected agent/model becomes inactive during wizard, block submit and ask user to reselect.
Cron invalid	Show inline error and sample expected patterns.
Backend validation failed	Show top summary and field-level errors returned by API.

Screen Acceptance Criteria

- Wizard must not allow submit if workflow, owner, schedule type or agent assignment is missing.
- Execution mode must be validated against the selected agent registry boundary.
- High-risk and write-capable tool selections must automatically set approval_required or show required approval reason.
- All dropdowns must be backend-driven and scoped by tenant/user permissions.

9. Schedule Detail Screen

Purpose: Provide one complete governed view of a schedule: configuration, lifecycle, agent/model assignment, execution boundaries, approvals, run history, notifications and audit timeline.

Item	Specification
Route	/workflow-scheduler/:id
Primary users	Schedule owner, Governance Admin, AI Asset Owner, Risk Manager, Compliance Officer, Auditor
Core outcome	User can inspect, manage and audit a governed schedule without switching between multiple modules.
Primary backend dependency	GET /api/v1/workflow-scheduler/schedules/{id}
Key governance rule	Sensitive sections and actions must be filtered by backend authorization and ABAC scope.

Recommended Layout

- Header: schedule name, code, status badge, risk badge, owner, next run, last run, primary actions.
- Tabs: Overview, Agent Assignment, Boundaries, Schedule Timing, Approvals, Runs, Audit Timeline, Change History.
- Overview tab should summarize workflow, purpose, current status, risk, approval state and health.
- Run summary card should show last run status, duration, outputs count and failure count.
- Audit Timeline tab must be read-only and chronological.

Fields, Controls and Data Binding

UI Area / Field	Control Type	Source / API	Validation / Behavior
Header Status	Badge	schedule_status	Use DRAFT/PENDING_APPROVAL/ACTIVE/PAUSED/RETIRED.
Owner	Read-only user card	owner_user_id	Show department and contact if available.
Approval State	Banner/card	workflow_schedule_approvals	Show pending/approved/rejected and reason.
Agent Assignment	Panel/table	schedule_agent_assignments	Show agent, model, execution mode, allowed tools/data.
Timing	Card	schedule_fields	Show schedule type, cron, timezone, next/last run.
Run History Preview	Table	workflow_runs	Latest 5 with link to full run history.
Audit Timeline	Timeline	audit.audit_events	Show event code, actor, time, summary.

Actions and Button Rules

Action	Visible To	Backend Call	Enable / Disable Rule	Result
Submit for Approval	SUBMIT_WORKFLOW_SCHEDULE	POST /submit	Schedule DRAFT and validation complete.	PENDING_APPROVAL.
Activate	ACTIVATE_WORKFLOW_SCHEDULE	POST /activate	Approved or approval not required.	ACTIVE.
Run Now	RUN_WORKFLOW_SCHEDULE	POST /run-now	ACTIVE and allowed.	Run queued.
Pause	PAUSE_WORKFLOW_SCHEDULE	POST /pause	ACTIVE.	PAUSED.
Resume	RESUME_WORKFLOW_SCHEDULE	POST /resume	PAUSED.	ACTIVE.
Retire	RETIRE_WORKFLOW_SCHEDULE	POST /retire	Not terminal.	RETIRED.
Edit	UPDATE_WORKFLOW_SCHEDULE	PUT schedule	DRAFT or PAUSED; active edits may require approval.	Updated.

UI States

State	User Experience
Pending approval	Show approval banner and disable activation for unauthorized users.
Retired	Disable all lifecycle actions except view history.
Failed last run	Show failure card with link to run detail.
Partial visibility	Hide restricted output and show reason: output is restricted by sensitivity or ABAC scope.

Screen Acceptance Criteria

- Schedule Detail must expose enough information to reconstruct why a schedule is allowed to run.
- All lifecycle buttons must reflect backend permission and schedule status.
- Audit timeline must show schedule creation, submission, approval, activation, pauses, resumes, retirements and manual runs.

10. Run History Screen

Purpose: Provide searchable operational history of scheduled, manual, event-triggered and API-triggered workflow runs.

Item	Specification
Route	/workflow-runs
Primary users	Governance Admin, AI Reviewer, Risk Manager, Compliance Officer, Auditor, Schedule Owner
Core outcome	User can quickly locate runs by workflow, schedule, status, risk, date range, trigger type and failure condition.
Primary backend dependency	GET /api/v1/workflow-runs
Key governance rule	Users must only see runs for schedules they are authorized to view.

Recommended Layout

- Top filters: run status, risk level, trigger type, date range, workflow, schedule, agent, owner.
- Run table with run code, schedule, workflow, trigger type, status, start/completed time, duration, risk and output count.
- Quick filters: Failed, Running, High Risk, SLA Breached, Manual Runs, Today.
- Export button only for auditor/admin if permitted; export must respect masking rules.

Fields, Controls and Data Binding

UI Area / Field	Control Type	Source / API	Validation / Behavior
Run Status	Filter	workflow_runs.run_status	QUEUED/RUNNING/COMPLETED/FAILED/CANCELLED/SKIPPED.
Trigger Type	Filter	workflow_runs.trigger_type	SCHEDULED/MANUAL/EVENT/API.
Date Range	Date picker	created_at/started_at	Default: last 7 days.
Run Table	Data table	workflow_runs	Server-side pagination required.
Risk Level	Badge	workflow_runs.risk_level	High/Critical highlighted.

Actions and Button Rules

Action	Visible To	Backend Call	Enable / Disable Rule	Result
Open Run	VIEW_WORKFLOW_RUN	GET /workflow-runs/{id}	User can view run.	Run Detail opens.
Cancel Run	CANCEL_WORKFLOW_RUN	POST /cancel	Run is RUNNING and user authorized.	Run cancelled.
Retry Failed Run	RUN_WORKFLOW_SCHEDULE or admin	POST retry endpoint future	Failed and retry allowed.	Retry queued.
Export	AUDIT/EXPORT permission	GET export future	User has export and scope.	Download generated.

UI States

State	User Experience
No runs	Show empty state with explanation and link to active schedules.
Running runs	Show live/refresh indicator; allow manual refresh.
Failed runs	Show failure type and escalation status.

Screen Acceptance Criteria

- Run History must be searchable and pageable.
- Failed and high-risk runs must be immediately visible.
- Run visibility must be backend-authorized and not merely frontend-filtered.

11. Run Detail Screen

Purpose: Show end-to-end execution trace of a workflow run: trigger, schedule, agent boundary check, model invocation, outputs, recommendations, evidence, failures and audit events.

Item	Specification
Route	/workflow-runs/:runId
Primary users	Auditor, Governance Admin, AI Reviewer, Risk Manager, Compliance Officer, Schedule Owner
Core outcome	User can understand exactly what happened during the run, why it passed/failed, what was generated, and what evidence supports it.
Primary backend dependency	GET /api/v1/workflow-runs/{run_id}, /steps, /outputs
Key governance rule	Raw output and evidence visibility must be ABAC-controlled by output sensitivity and user scope.

Recommended Layout

- Header: run code, status, risk, schedule, workflow, trigger type, duration, started/completed timestamps.
- Left or top timeline: QUEUED, STARTED, BOUNDARY_CHECK, AGENT_EXECUTION, OUTPUT_GENERATED, COMPLETED/FAILED.
- Tabs: Summary, Steps, Outputs, Findings, Recommendations, Evidence, Failures, Audit Events.
- Output tab must show structured output first and raw output only if authorized.
- Failure tab must show failure type, message, retry count, escalation status and next action.

Fields, Controls and Data Binding

UI Area / Field	Control Type	Source / API	Validation / Behavior
Run Header	Read-only cards	workflow_runs	Display status, risk, trigger, timing.
Run Timeline	Timeline	workflow_run_steps + audit events	Show step status and duration.
Outputs	Structured viewer	workflow_run_outputs	Findings, recommendations, evidence, risk summary.
Raw Output	JSON viewer	workflow_run_outputs metadata/raw	Only if authorized.
Failure Details	Panel	workflow_run_failures	Visible if run failed or warning.
Audit Events	Timeline	audit.audit_events	Chronological immutable audit trail.

Actions and Button Rules

Action	Visible To	Backend Call	Enable / Disable Rule	Result
Cancel	CANCEL_WORKFLOW_RUN	POST /cancel	Run RUNNING and authorized.	Cancelled.
Retry	Authorized owner/admin	POST retry future	Run failed and retry allowed.	Retry queued.
Create Review Task	Risk/Compliance/Admin	POST notification/task future	High-risk output.	Task created.
Download Evidence	AUDIT/EXPORT	GET evidence future	Authorized and evidence not restricted.	Download.

UI States

State	User Experience
Completed	Show green completed summary and outputs.
Failed	Show failure summary, failed step and recovery options.
Boundary violation	Highlight blocked action and tool/data boundary rule.
Output restricted	Mask raw/sensitive content and show access message.

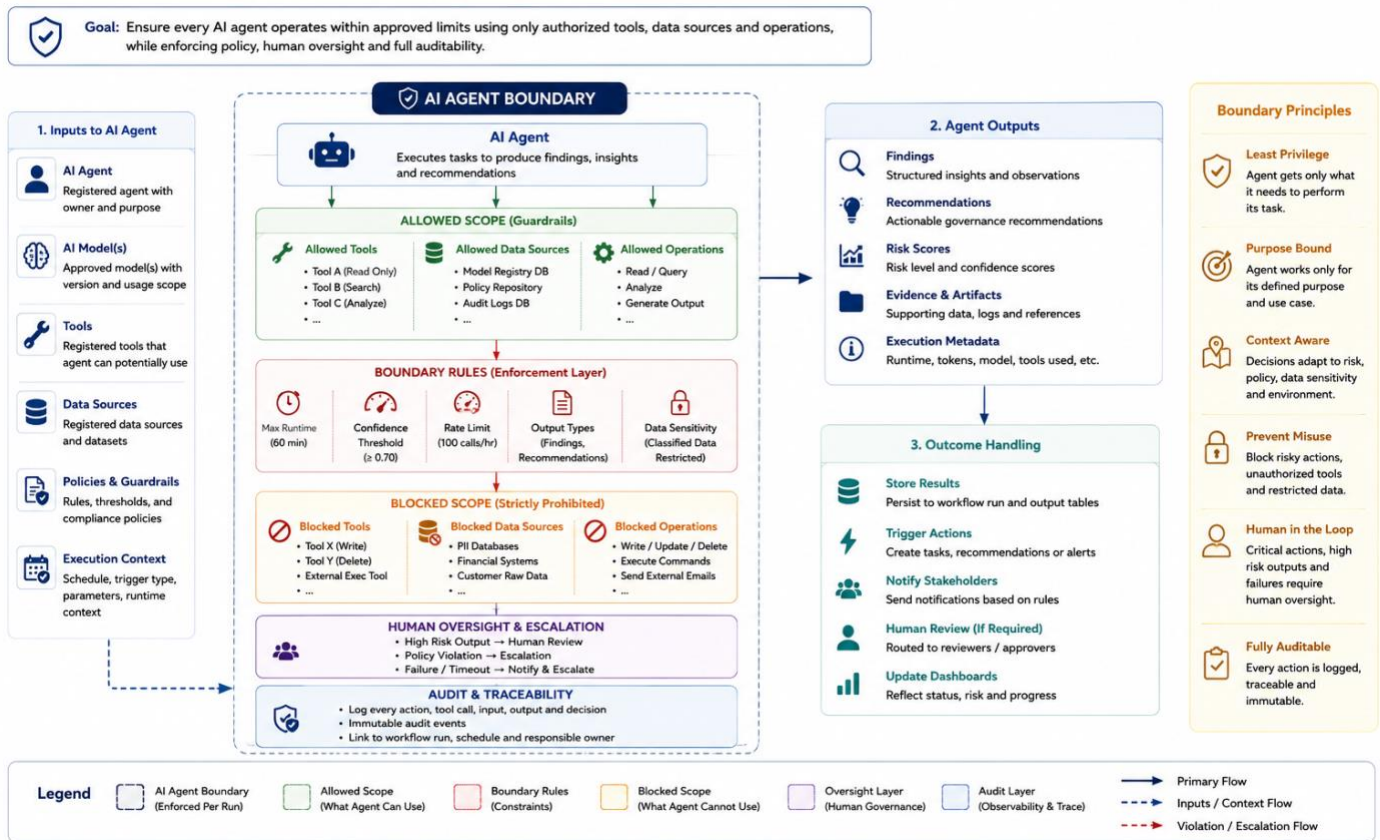
Screen Acceptance Criteria

- Run Detail must show every run step with timestamp and status.
- High-risk findings must be clearly visible and linked to escalation/notification if created.
- Audit event timeline must support reconstruction of the run lifecycle.

12. Agent Assignment Matrix

5. AI Agent Boundary Model

Defining Controlled Execution Boundaries for Safe, Governed and Accountable AI Agent Operations



Purpose: Allow authorized users to view and manage which AI agents, models, tools and data sources are assigned to which governed schedules and workflows.

Item	Specification
Route	/agent-assignments
Primary users	Governance Admin, AI Asset Owner, System Admin
Core outcome	Creates transparent relationship matrix across workflow, schedule, AI agent, AI model, tools and execution boundaries.
Primary backend dependency	GET/POST/PUT workflow_schedule_agent_assignments
Key governance rule	Assignment cannot exceed registered agent/model/tool boundary from Phase 1 registry.

Recommended Layout

- Matrix table: Workflow, Schedule, Agent, Model, Execution Mode, Allowed Tools, Approval Required, Status.
- Filters by workflow, agent, model, execution mode, risk level and approval requirement.
- Edit drawer for boundary configuration and assignment role.
- Boundary warnings when selected tools are write-capable or restricted.

Fields, Controls and Data Binding

UI Area / Field	Control Type	Source / API	Validation / Behavior
Workflow/Schedule	Dropdown/filter	GET schedules	Only active/draft schedules based on edit mode.
Agent	Dropdown	GET registry agents	Must be active.

Model	Dropdown	GET registry models	Must be active and allowed for agent if constrained.
Execution Mode	Dropdown	lookup + agent boundary	Cannot exceed registry boundary.
Allowed Tools	Multi-select	GET tools	Write/execute tools require approval.
Allowed Data Sources	Multi-select	GET data sources	Restricted data triggers warning/approval.
Blocked Operations	Tag input	lookup	Recommended for all non-read-only assignments.

Actions and Button Rules

Action	Visible To	Backend Call	Enable / Disable Rule	Result
Add Assignment	ASSIGN_AI_AGENT_TO_WORKFLOW	POST assignment	Schedule editable and user authorized.	Assignment saved.
Edit Assignment	UPDATE permission	PUT assignment	Assignment active/draft.	Updated.
Disable Assignment	UPDATE permission	PUT status	Assignment active.	Inactive.
Validate Boundary	Authorized user	POST authorization/evaluate or validation	Any assignment edit.	Pass/fail shown.

UI States

State	User Experience
Boundary valid	Show validation success with allowed operations summary.
Boundary invalid	Show exact reason: inactive tool, execution exceeds agent, restricted source, write tool approval missing.
Read-only mode	Auditors see matrix without edit actions.

Screen Acceptance Criteria

- No assignment can be saved if execution mode exceeds registered agent boundary.
- Allowed tools and data sources must be explicitly selected or inherited through clearly visible default rules.
- Boundary validation results must be visible before saving.

13. Authorization Simulator

Purpose: Help administrators and developers test whether a user or AI agent can perform an action on a schedule, run or output under a specific RBAC + ABAC context.

Item	Specification
Route	/authorization-simulator
Primary users	System Admin, Governance Admin, Security/Admin, Architect, QA
Core outcome	Provides transparent Allow/Deny decision with reason, evaluated attributes and relationship checks.
Primary backend dependency	POST /api/v1/authorization/evaluate
Key governance rule	Simulator must not grant access; it only evaluates and records/optionally persists a decision.

Recommended Layout

- Three-column input layout: Subject, Object, Action/Environment.
- Subject: user/agent, roles, department, business unit, delegation state.
- Object: schedule/run/output/workflow, risk, department, sensitivity, owner.
- Action: create, activate, run now, view output, cancel, override.
- Decision result panel: Allowed/Denied, reasons, rule details, evaluated attributes.

Fields, Controls and Data Binding

UI Area / Field	Control Type	Source / API	Validation / Behavior
Subject Type	Dropdown	USER/AGENT/SERVICE	Required.
Subject	Search dropdown	users/agents API	Required.
Object Type	Dropdown	enum	SCHEDULE/RUN/OUTPUT/ASSIGNMENT.
Object	Search dropdown	object APIs	Required if persisted object exists.
Action	Dropdown	permission/action enum	Required.
Environment	Checkboxes/fields	form	Emergency flag, delegation active, current time.
Decision Output	Panel	authorization response	Allow/Deny + reasons.

Actions and Button Rules

Action	Visible To	Backend Call	Enable / Disable Rule	Result
Evaluate	EVALUATE_AUTHORIZATION	POST /authorization/evaluate	All required inputs present.	Decision displayed.
Reset	Simulator user	client action	Any time.	Form cleared.
Copy Payload	Simulator user	client action	Decision exists.	JSON copied.

UI States

State	User Experience
Allowed	Green decision card with rule evidence.
Denied	Red/amber decision card with missing RBAC/ABAC/relationship reason.
Incomplete context	Prompt user to select subject, object and action.

Screen Acceptance Criteria

- Simulator must show RBAC, ABAC and relationship-check sections separately.
- Denied decisions must be explainable and usable by developers for debugging.
- Simulator must be clearly marked as evaluation only and not an approval action.

14. Schedule Approval Queue

Purpose: Allow authorized approvers and approval group members to review schedules requiring activation approval or high-risk change approval.

Item	Specification
Route	/schedule-approvals
Primary users	Risk Manager, Compliance Officer, Business Approver, Approval Group Member, Governance Admin
Core outcome	Approvers can approve, reject, request changes or escalate with full context and audit trail.
Primary backend dependency	workflow_schedule_approvals APIs and schedule submit/activate APIs
Key governance rule	Approver must be valid approval group member, assigned approver, delegated approver or authorized escalation owner.

Recommended Layout

- Queue list with tabs: Pending My Approval, Group Queue, Escalated, Completed.
- Approval detail drawer/page with schedule summary, risk reasons, agent boundaries, tools/data, owner and change history.
- Decision section: Approve, Reject, Request Changes, Escalate. Reason required for reject/escalate.
- Audit context panel showing who submitted, when, why approval required and previous decisions.

Fields, Controls and Data Binding

UI Area / Field	Control Type	Source / API	Validation / Behavior
Approval Item	Table row/card	workflow_schedule_approvals	Show schedule, risk, owner, submitted date, SLA.
Approval Reason	Read-only text	approval_reason	Must explain high risk/write tool/restricted source.
Decision Reason	Textarea	form	Required for reject, request changes, escalate; optional for approve.
Approver Identity	Read-only	backend auth context	Show acting as self/delegate/group member.
Schedule Summary	Read-only panel	schedule detail	Show workflow, agent, model, tools, data, timing.

Actions and Button Rules

Action	Visible To	Backend Call	Enable / Disable Rule	Result
Approve	ACTIVATE/APPROVE permission	POST approval approve	User authorized and item pending.	Approved and possibly active.
Reject	APPROVE permission	POST reject	Reason required.	Rejected.
Request Changes	APPROVE/REVIEW	POST request changes	Reason required.	Returned to owner.
Escalate	ESCALATE permission	POST escalate	Escalation owner configured.	Escalated.

UI States

State	User Experience
No pending approvals	Show empty state and quick link to scheduler dashboard.
Delegated approval	Show banner: You are acting under delegation from original approver.
SLA breach	Show overdue badge and escalation recommendation.

Screen Acceptance Criteria

- Approver must see enough context to make decision without opening database/admin tools.
- Reject, request changes and escalate must require decision reason.
- Approval decisions must generate audit events and be visible on Schedule Detail.

15. Workflow Notifications Center

Purpose: Provide an in-app center for schedule approval requests, activation decisions, failed runs, SLA breaches, high-risk outputs and authorization denials.

Item	Specification
Route	/workflow-notifications
Primary users	All users with assigned notifications
Core outcome	Users can see and act on governance notifications relevant to them.
Primary backend dependency	GET workflow_notifications, PUT read/acknowledge future
Key governance rule	Notification visibility must be restricted to recipient, group member or authorized admin/auditor.

Recommended Layout

- Tabs: Unread, Action Required, Failures, High Risk, All.
- Notification cards with type, severity, title, related entity, created time, status and action link.
- Bulk mark as read only for non-action-required notifications.
- Action buttons route user to approval queue, run detail or schedule detail.

Fields, Controls and Data Binding

UI Area / Field	Control Type	Source / API	Validation / Behavior
Notification Type	Badge	notification_type	Approval, Failure, SLA, High Risk, Authorization Denied.
Severity	Badge	severity	Info/Low/Medium/High/Critical.
Title/Message	Text	title/message	Concise and action-oriented.
Related Entity	Link	entity_type/entity_id	Open schedule/run/approval detail.
Status	Badge	status	Unread/Read/Acknowledged/Closed.

Actions and Button Rules

Action	Visible To	Backend Call	Enable / Disable Rule	Result
Open Related Item	Authorized recipient	navigate	Related item exists and visible.	Detail opens.
Mark as Read	Recipient	PUT notification	Not action required.	Read.
Acknowledge	Recipient	PUT acknowledge	Allowed for assigned notification.	Acknowledged.

UI States

State	User Experience
No notifications	Show calm empty state.
Action required	Highlight with strong visual priority and link to decision screen.
Restricted related item	Show notification but hide details if object no longer visible.

Screen Acceptance Criteria

- Notifications must route users to the correct work screen.
- High-risk and SLA breach notifications must be visually prioritized.
- A user must not see notifications outside their recipient/group/admin scope.

16. Design Patterns for Key User Journeys

16.1 Journey A: Create and Submit a High-Risk Schedule

1. User opens Workflow Scheduler Dashboard and clicks Create Schedule.
2. User selects an ACTIVE registered workflow from Phase 1 registry.
3. User assigns ACTIVE AI agent and model.
4. User selects execution mode and allowed tools/data sources.
5. UI detects high risk or write/restricted tool and shows approval required banner.
6. User configures timing, concurrency, retry policy and timezone.
7. User selects owner, reviewer and approval group.
8. Review screen shows validation results and approval reason.
9. Submit creates DRAFT or PENDING_APPROVAL schedule depending backend decision.
10. Audit event is generated and approval queue receives item.

16.2 Journey B: Approve and Activate Schedule

11. Approver opens Schedule Approval Queue.
12. Approver reviews schedule summary, risk, agent boundary, tools, data sources and approval reason.
13. Approver clicks Approve, Reject, Request Changes or Escalate.
14. Backend validates RBAC + ABAC + group/delegation relationship.
15. Decision is persisted and audit event generated.
16. If approved and activation rules are complete, schedule becomes ACTIVE.
17. Schedule owner receives notification.

16.3 Journey C: Run Execution and Audit Review

18. Scheduler worker or user triggers active schedule.
19. Run is created in QUEUED/RUNNING state.
20. Run detail starts showing execution steps.
21. Agent boundary check is performed before model/tool invocation.
22. Outputs are parsed into findings, recommendations and evidence.
23. Run status becomes COMPLETED or FAILED.
24. Audit events are visible in Run Detail and Schedule Detail.
25. If high-risk output appears, notification and review task are created.

17. UI-to-API Mapping

Screen	Operation	API	Method	Permission
Scheduler Dashboard	List schedules	/api/v1/workflow-scheduler/schedules	GET	VIEW_WORKFLOW_SCHEDULE
Create Wizard	Create schedule	/api/v1/workflow-scheduler/schedules	POST	CREATE_WORKFLOW_SCHEDULE
Schedule Detail	Get schedule	/api/v1/workflow-scheduler/schedules/{id}	GET	VIEW_WORKFLOW_SCHEDULE
Schedule Detail	Update schedule	/api/v1/workflow-scheduler/schedules/{id}	PUT	UPDATE_WORKFLOW_SCHEDULE
Schedule Detail	Submit	/api/v1/workflow-scheduler/schedules/{id}/submit	POST	SUBMIT_WORKFLOW_SCHEDULE
Schedule Detail	Activate	/api/v1/workflow-scheduler/schedules/{id}/activate	POST	ACTIVATE_WORKFLOW_SCHEDULE
Schedule Detail	Pause	/api/v1/workflow-scheduler/schedules/{id}/pause	POST	PAUSE_WORKFLOW_SCHEDULE
Schedule Detail	Resume	/api/v1/workflow-scheduler/schedules/{id}/resume	POST	RESUME_WORKFLOW_SCHEDULE
Schedule Detail	Retire	/api/v1/workflow-scheduler/schedules/{id}/retire	POST	RETIRE_WORKFLOW_SCHEDULE
Schedule Detail	Run now	/api/v1/workflow-scheduler/schedules/{id}/run-now	POST	RUN_WORKFLOW_SCHEDULE
Run History	List runs	/api/v1/workflow-runs	GET	VIEW_WORKFLOW_RUN
Run Detail	Get run	/api/v1/workflow-runs/{run_id}	GET	VIEW_WORKFLOW_RUN
Run Detail	Get steps	/api/v1/workflow-runs/{run_id}/steps	GET	VIEW_WORKFLOW_RUN
Run Detail	Get outputs	/api/v1/workflow-runs/{run_id}/outputs	GET	VIEW_WORKFLOW_RUN_OUTPUT

Run Detail	Cancel	/api/v1/workflow-runs/{run_id}/cancel	POST	CANCEL_WORKFLOW_RUN
Authorization Simulator	Evaluate	/api/v1/authorization/evaluate	POST	EVALUATE_AUTHORIZATION

18. Data Binding and Frontend Models

```
// TypeScript interface examples
export interface WorkflowScheduleListItem {
  id: string;
  schedule_code: string;
  schedule_name: string;
  workflow_name: string;
  schedule_type: 'MANUAL' | 'ONE_TIME' | 'INTERVAL' | 'DAILY' | 'WEEKLY' | 'MONTHLY' | 'CRON';
  schedule_status: 'DRAFT' | 'PENDING_APPROVAL' | 'ACTIVE' | 'PAUSED' | 'RETIRED' | 'FAILED';
  risk_level: 'LOW' | 'MEDIUM' | 'HIGH' | 'CRITICAL';
  owner_name: string;
  next_run_at?: string;
  last_run_at?: string;
  approval_required: boolean;
  health_status: 'HEALTHY' | 'ATTENTION' | 'FAILED' | 'SLA_BREACHED';
}

export interface ScheduleWizardForm {
  workflow_id: string;
  schedule_code: string;
  schedule_name: string;
  schedule_description?: string;
  agent_assignments: AgentAssignmentForm[];
  schedule_type: string;
  cron_expression?: string;
  timezone: string;
  start_at?: string;
  end_at?: string;
  concurrency_policy: string;
  retry_policy: { max_retries: number; retry_delay_seconds: number };
  owner_user_id: string;
  approval_required: boolean;
  approval_group_id?: string;
  risk_level: string;
}

export interface AgentAssignmentForm {
  agent_id: string;
  model_id?: string;
  assignment_role: 'PRIMARY' | 'SECONDARY' | 'REVIEWER';
  execution_mode: 'READ_ONLY' | 'RECOMMEND_ONLY' | 'APPROVAL_REQUIRED' | 'LIMITED_EXECUTION';
  confidence_threshold?: number;
  allowed_tools: string[];
  allowed_data_sources: string[];
  blocked_operations: string[];
  boundary_rules: Record<string, unknown>;
}
```

19. Permission-Driven UI Behavior

Permission / Context	Frontend Behavior	Backend Requirement
VIEW_WORKFLOW_SCHEDULE missing	Do not show scheduler menu or show unauthorized state.	Reject GET with 403 if called.
CREATE_WORKFLOW_SCHEDULE missing	Hide Create Schedule button.	Reject POST create with 403.
ACTIVATE_WORKFLOW_SCHEDULE missing	Hide or disable Activate button.	Reject activate with authorization reason.
High-risk schedule + not approval group member	Show approval requirement but no approve button.	Deny approval unless delegation valid.

VIEW_WORKFLOW_RUN_OUTPUT missing	Show run summary but hide output tab or mask content.	Do not return sensitive output payload.
CANCEL_WORKFLOW_RUN missing	Hide Cancel button.	Reject cancel with 403.
Auditor role	Read-only screens; hide mutation buttons.	Reject write APIs.

20. Validation Rules for Frontend Developers

Rule ID	Validation Rule	Frontend Timing	Backend Must Revalidate
VAL-001	Workflow is required and must be active.	Wizard Step 1	Yes
VAL-002	Schedule name and code are required.	Wizard Step 1/Review	Yes
VAL-003	At least one AI agent assignment is required for executable schedules.	Wizard Step 2	Yes
VAL-004	Selected agent must be active.	Wizard Step 2	Yes
VAL-005	Execution mode cannot exceed agent registry execution mode.	Wizard Step 2	Yes
VAL-006	Confidence threshold must be between 0 and 100.	Wizard Step 2	Yes
VAL-007	Write or execute tools require approval_required=true.	Wizard Step 3	Yes
VAL-008	Restricted data source requires approval group.	Wizard Step 3/5	Yes
VAL-009	Cron expression required and valid when schedule type is CRON.	Wizard Step 4	Yes
VAL-010	End date must be after start date.	Wizard Step 4	Yes
VAL-011	Owner is required.	Wizard Step 5	Yes
VAL-012	High/Critical schedules require reviewer or approval group.	Wizard Step 5	Yes

21. Accessibility and Usability Requirements

- All form fields must have visible labels, not only placeholders.
- Color must not be the only indicator of risk or status; use text labels and icons.
- Tables must support keyboard navigation and accessible row actions.
- Modals must trap focus and close with Esc only if no required confirmation is pending.
- Error messages must be placed near fields and summarized at top of wizard step.
- Long JSON output must be collapsible and copyable, with line wrapping for readability.
- Status badges must have accessible text equivalents.
- Audit timelines must be readable in chronological order and screen-reader friendly.

22. QA Checklist for UI Implementation

Area	Checklist
Scheduler Dashboard	Filters work, pagination works, row actions obey permissions, high-risk and failed records are visible.
Create Wizard	All six steps validate correctly; approval banner appears for high risk, write tools and restricted data.
Schedule Detail	Tabs load correct data; lifecycle actions update status; audit timeline appears.
Run History	Filters/search work; failed/running/high-risk runs are easy to identify.
Run Detail	Timeline, outputs, failures and audit events display correctly; restricted output is masked.
Agent Assignment Matrix	Boundary validation prevents unsafe assignments.
Authorization Simulator	Allow/deny reasons display separately for RBAC, ABAC and relationship checks.
Approval Queue	Approve/reject/request changes/escalate require correct authorization and reason where needed.
Notifications	Action links navigate correctly; visibility is scoped.
Cross-cutting	Loading, empty, error, unauthorized and partial-access states exist on all screens.

23. Frontend Implementation Sequence

Sequence	Work Item	Dependencies	Expected Output
1	Create route skeleton and sidebar menu entries	Routing framework	All Phase 2 pages reachable in dev.
2	Build shared API client and response envelope handling	Backend API standards	Consistent success/error handling.

3	Build common components: badges, tables, filters, modals, audit panel	Design system	Reusable UI foundation.
4	Build Scheduler Dashboard	Schedules list API	List/filter/action screen.
5	Build Create Schedule Wizard	Registry dropdown APIs + schedule create API	Guided schedule creation.
6	Build Schedule Detail	Schedule detail, runs, approvals, audit APIs	Unified schedule view.
7	Build Run History and Run Detail	Run APIs	Execution traceability screens.
8	Build Approval Queue	Approval APIs	Human oversight screen.
9	Build Authorization Simulator	Authorization evaluate API	Debug/validation tool.
10	Build Notifications Center	Notifications API	User notification workspace.
11	Implement permission gating and ABAC result handling	Auth decision APIs	Buttons/sections obey access scope.
12	QA hardening and demo seed validation	Seed data	Demo-ready Phase 2 UI.

24. Developer Handoff Notes

Frontend rule: Do not hardcode workflow, agent, model, tool, data source, owner, approval group, risk or status options except temporary local mocks during early development. The final implementation must use backend-driven registries and lookups.

Security rule: Frontend permission gating improves usability but is not security. Every write action and sensitive read must be authorized by backend RBAC + ABAC + relationship checks.

Audit rule: Schedule Detail and Run Detail must always show audit visibility where the user is authorized. If audit events are missing for critical lifecycle actions, QA should fail the story.

25. Appendix A - Sample Create Schedule Payload

```
{
  "workflow_id": "uuid-of-registered-workflow",
  "schedule_code": "DAILY_MODEL_RISK_REVIEW",
  "schedule_name": "Daily AI Model Risk Review",
  "schedule_type": "DAILY",
  "timezone": "Asia/Kolkata",
  "start_at": "2026-06-17T09:00:00+05:30",
  "concurrency_policy": "SKIP_IF_RUNNING",
  "max_runtime_seconds": 1800,
  "owner_user_id": "uuid-owner",
  "approval_required": true,
  "approval_group_id": "uuid-ai-governance-board",
  "risk_level": "HIGH",
  "agent_assignments": [
    {
      "agent_id": "uuid-agent",
      "model_id": "uuid-model",
      "execution_mode": "RECOMMEND_ONLY",
      "confidence_threshold": 80,
      "allowed_tools": ["REGISTRY_READ_API", "AUDIT_READ_API"],
      "allowed_data_sources": [],
      "blocked_operations": ["UPDATE_POLICY", "EXECUTE_EXTERNAL_ACTION"],
      "boundary_rules": {
        "max_records": 500,
        "allow_write_tools": false,
        "requires_human_approval_for_high_risk": true
      }
    }
  ]
}
```


26. Appendix B - Final Definition of Done for UI

- All Phase 2 routes are implemented and connected to backend APIs.
- Create Schedule Wizard supports all six required steps and prevents unsafe configuration.
- Schedule Dashboard, Detail, Run History, Run Detail, Approval Queue and Authorization Simulator are functional.
- All actions use standard confirmation, error and success patterns.
- All sensitive actions are permission-gated in UI and backend-authorized.
- Run Detail displays run steps, outputs, failures and audit timeline.
- Approval Queue supports approve, reject, request changes and escalate with reason handling.
- All screens support loading, empty, error, unauthorized and partial-access states.
- At least one seeded demo journey works end to end: create schedule -> submit/approve -> activate -> run now -> view run detail -> audit timeline.